Unit-1

Introduction to C# and .NET Framework

Introduction to .NET Framework

software development environment developed by Microsoft for building & running application on windows.

Offers a **runtime environment**, **framework**, **libraries** & tools for developing and running application build in multiple language[C#, F#, C++, VB .NET]

NET Framework supports more than 60 programming languages in which 11 are designed and developed by Microsoft.

Enables developer to create and run application on multiple platforms [web, desktop, mobile].

Application Frameworks:

ASP.NET: For building web applications and APIs.

Windows Forms: For creating desktop applications with a graphical user interface (GUI) for Windows.

WPF (Windows Presentation Foundation): For building rich desktop applications with advanced UIs on Windows.

Xamarin: For developing cross-platform mobile apps for iOS and Android using C#.

.NET MAUI: For building cross-platform desktop and mobile apps with a single codebase (replacing Xamarin).

Blazor: For building interactive web UIs with C# instead of JavaScript, running in the browser via WebAssembly or on the server.

Entity Framework: For object-relational mapping (ORM), simplifying database access in .NET applications.

Language Interoperability

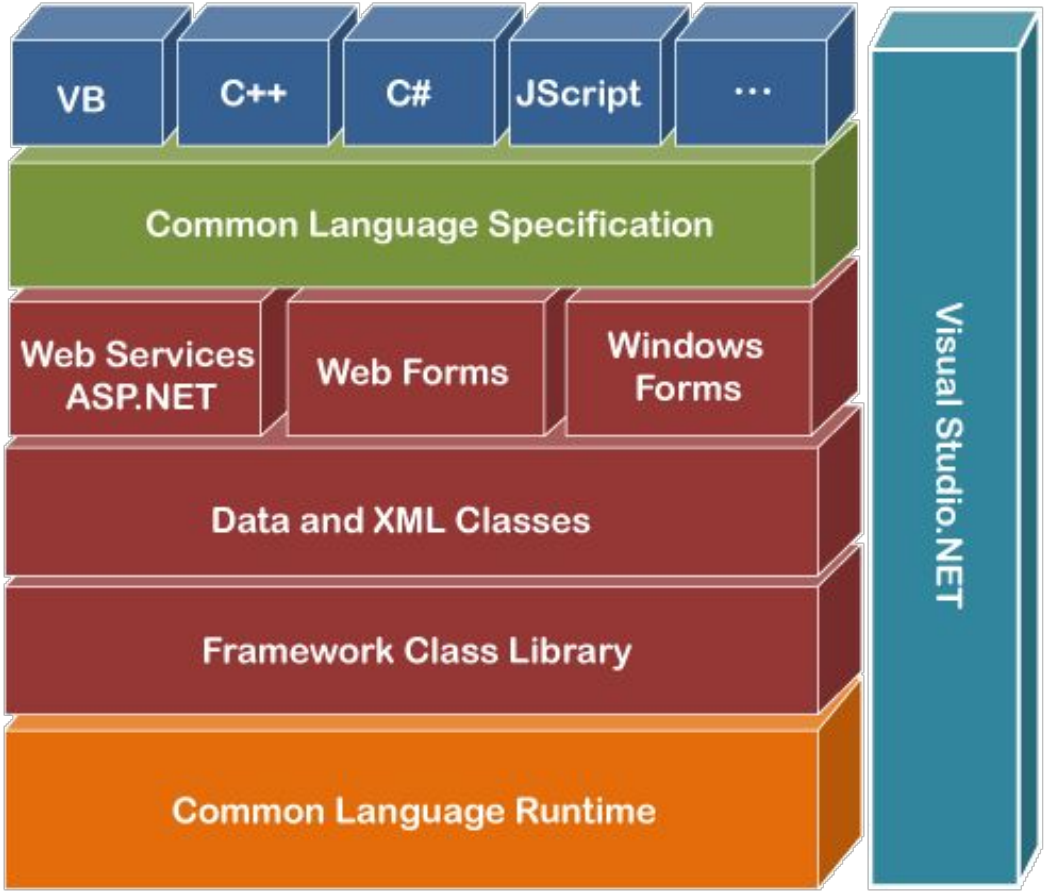
Allows developer to use multiple language like C#, F# and VB .NET with in same application.

Components written in different languages can interact seamlessly, enhancing flexibility and collaboration in developments projects.

MODERN Examples:

- Python.NET(pythonnet)
- C# & C++ (via P/Invoke)
- C# & Rust (via FFI)

Components of .NET Framework



Common Language Specification(CLS)

open specification and technical standard originally developed by Microsoft and standardized by ISO/IEC (ISO/IEC 23271) and Ecma International (ECMA 335)

a set of rules and guidelines that define how the .NET Framework's languages must behave and interact with each other.

responsible for converting the different .NET programming language syntactical rules and regulations into CLR understandable format

CLS Example

VB.NET has an integer data type
C# has an int data type for managing integers.

After compilation, Int32 is used by both data types. So, CLS provides the data types using managed code. A common type system helps in writing language-independent code.

Framework Class Library(FCL):-

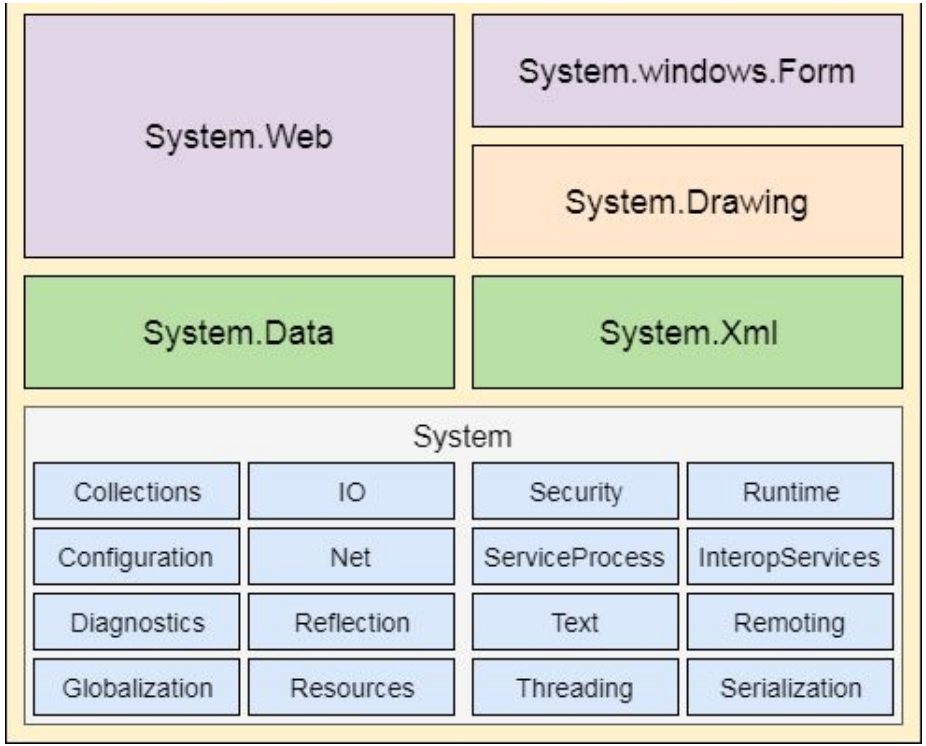
set of pre-built, reusable classes and APIs provided by the .NET Framework developers can use to build applications more efficiently

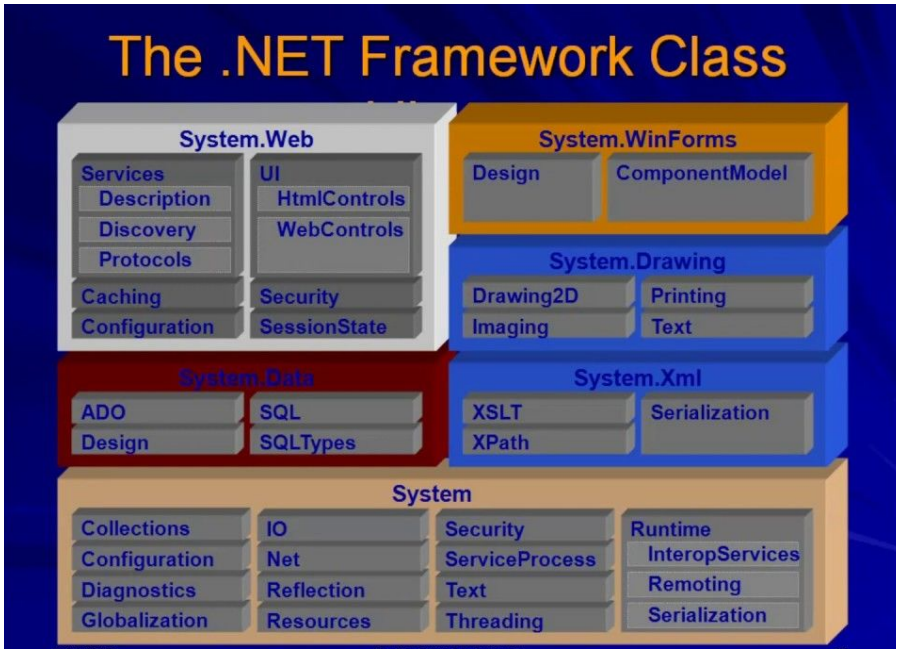
provides a huge collection of classes that cover a wide variety of functionality and domains

Base Class Library(BCL):-

sub set of FCL , provides the core set of libraries that are foundational to the .NET platform.

Fundamental & Basic classes

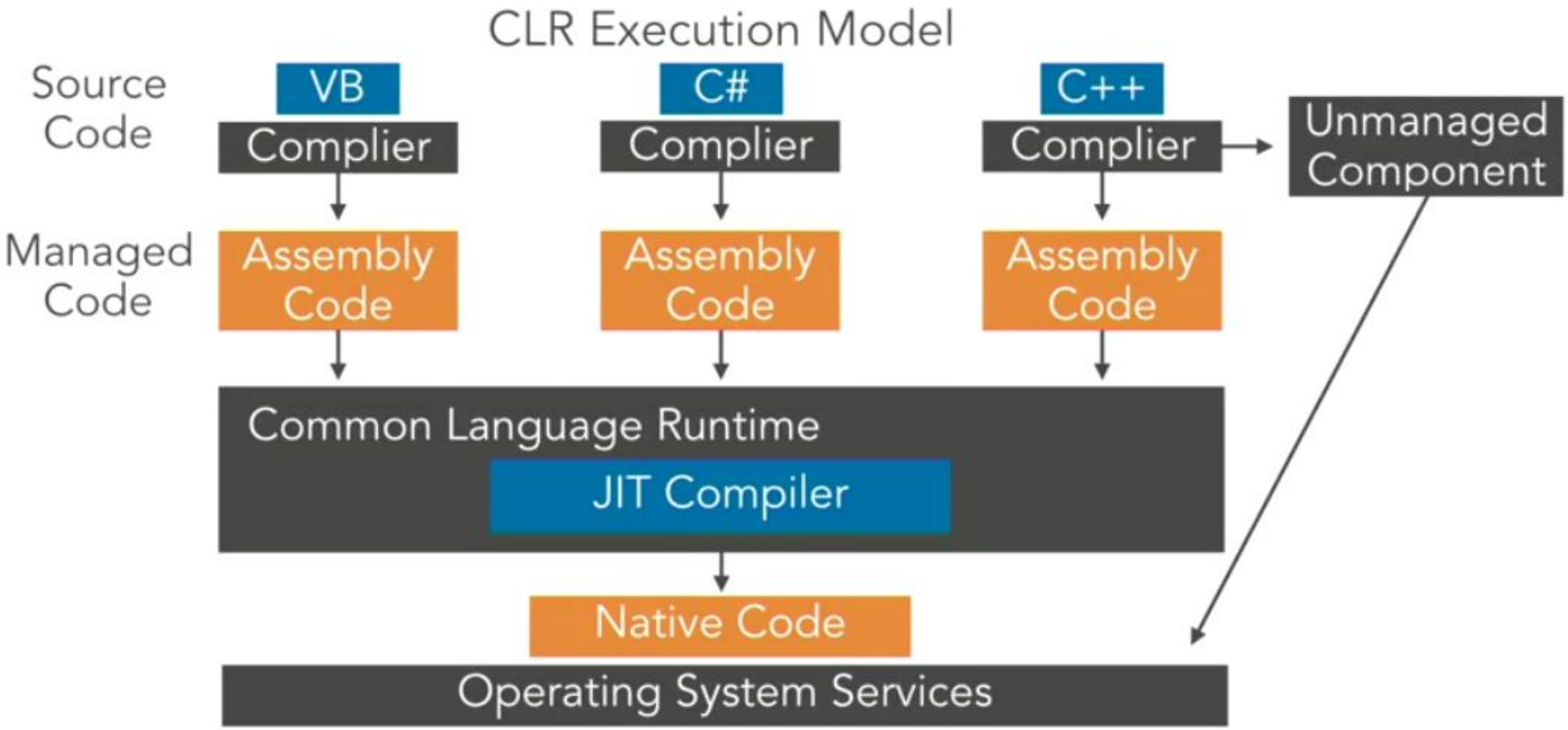




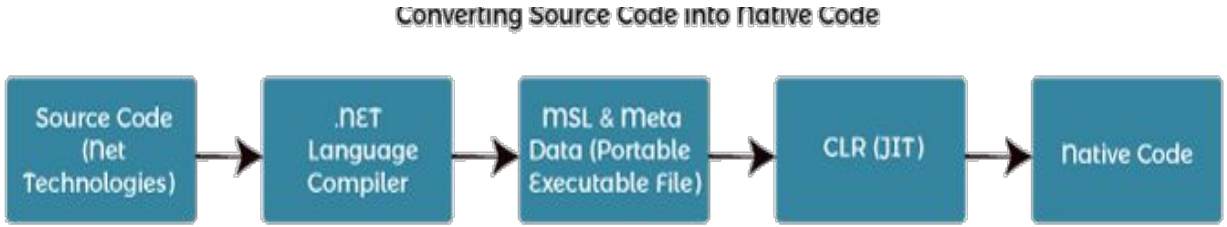
Managed vs Unmanaged Code

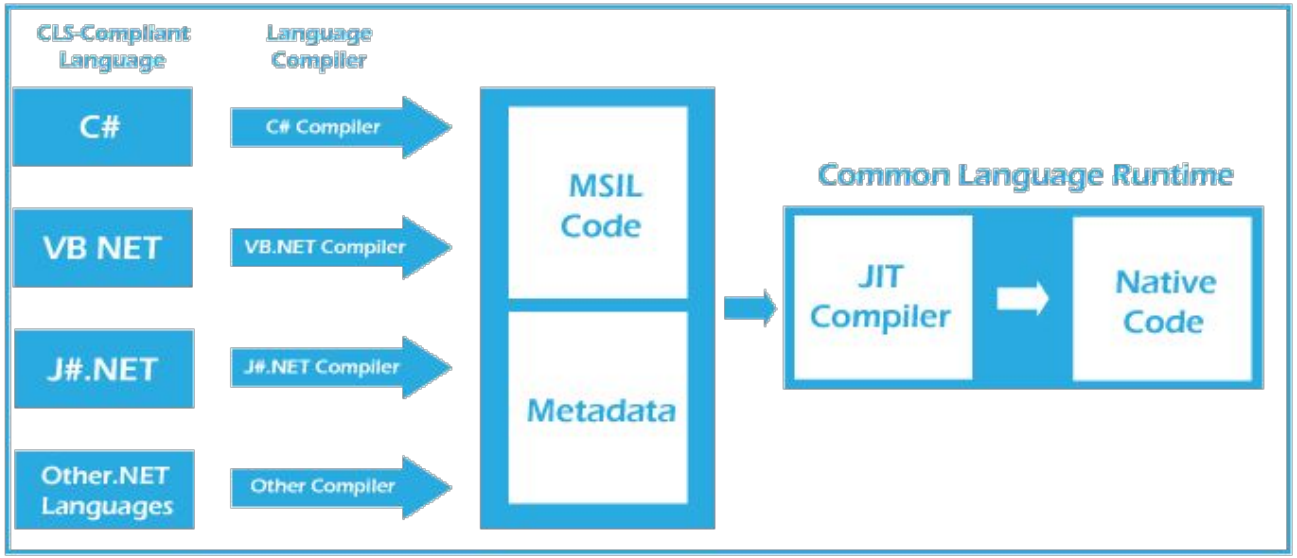
Unmanaged Code	Managed Code
Compilation: source code directly compiles into native languages.	**Compilation**: source code is compiled in the intermediate language known as IL or MSIL or CIL.
Execution: managed directly by the operating system.	**Execution**: managed runtime environment or managed by the CLR.
Runs: only on the specific platform for which it is compiled.	**Runs:** on any platform that has CLR installed.
CLR service: Automatic Garbage Collection, Security, Exception handling missing.	**CLR service:** Automatic Garbage Collection, Security, Exception handling present.
Example: C, C++	**Example:** C#, F#, VB Net

Managed & Unmanaged Code

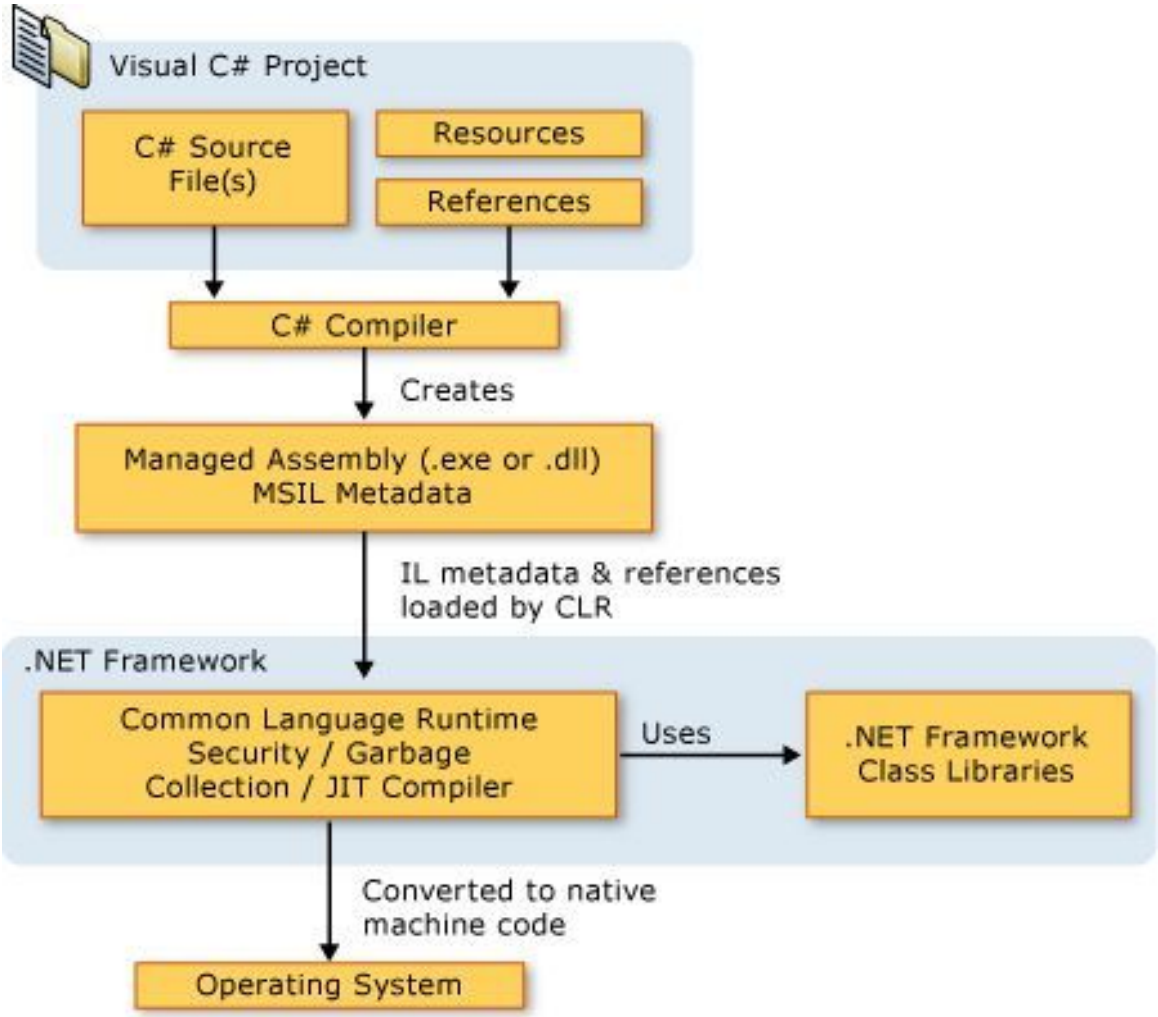


Execution of .NET Framework(Managed Code)





Exceution of a .NET Application


Visual C# Project
C# Source File(s)
Resources
References
C# Compiler
Creates
Managed Assembly (.exe or .dll)
MSIL Metadata
IL metadata & references loaded by CLR
.NET Framework
Common Language Runtime Security / Garbage Collection / JIT Compiler
Uses
.NET Framework Class Libraries
Converted to native machine code
Operating System

Language Compiler:

is a key tool that translates managed code into an intermediate language that the .NET runtime can execute.

Identifies syntax and semantic errors, enforces type safety, and ensures that code adheres to language rules, reducing runtime issues.

Popular language compiler:

- C# -> csc.exe
- VB .NET -> vbc.exe
- J# -> vjc.exe

C# Compiler

stands for the C# compiler in the .NET environment. It is a command-line tool provided by Microsoft to compile C# source code files into executable assemblies (EXE) or libraries (DLL).

Example:-

csc -optimize -debug -out:MyApp.exe Program.cs

csc /target:library /out:MyLibrary.dll MyLibrary.cs

MSIL(Microsoft Intermediate Language)

Low-level, human-readable programming language, platform-independent code generated by .NET language compilers.

(CLR) compiles MSIL to native code for the target platform using Just-In-Time (JIT) compilation.

Feature:-

Platform Independence: Not tied to a specific platform allows it to be executed on any platform that has a compatible Common Language Runtime (CLR) implementation

Language Interoperability: Code written in different .NET languages can interoperate seamlessly because they all compile down to MSIL

Assemblies:-

is a compiled unit in .NET that contains MSIL code along with metadata and other resources.

Assemblies are typically in the form of DLLs (Dynamic Link Libraries) or EXEs (executables).

are the deployable and reusable units in .NET that include all the information needed for runtime execution, including MSIL code, metadata, and resources.

MSIL, metadata, versioning, security settings, and resources.

CLR(Common Language Runtime)

runtime environment for the .NET Framework, responsible for managing the execution of .NET applications, providing essential services like memory management, security, and exception handling.

Main Components of CLR

- Code Execution with JIT
- Memory Management
- Security
- Runtime Exception Handling

Code Execution with JIT

Compilation to Machine Code: The JIT compiler converts Intermediate Language (IL) code into native machine code at runtime, making it executable by the target platform.

Platform-Specific Optimization: This process occurs just before execution, optimizing the code based on the specific platform and hardware for improved performance.

Memory Management

automates memory allocation and deallocation, ensuring efficient usage of resources while reducing the likelihood of memory leaks.

Techniques:

1. Managed Heap
2. Garbage Collection (GC)

Garbage Collection (GC)

The Garbage Collector automatically frees memory occupied by objects that are no longer used.

Steps in Memory Management via GC

1. Tracking Object References

- Local variables on the stack
- Static/Global variables
- CPU registers

Identifies reachable and unreachable objects

```
class Person { }
Person p1 = new Person();
Person p2 = new Person();
p1 = null;
```

p2 → still referenced by the program → reachable -> remains in memory

p1 → reference removed → unreachable -> collected by garbage collector

2. Marking Phase

The GC starts from the root references and recursively marks all reachable objects.

```
Root → A → C → D
B → E


Marked: A, C, D
Unmarked (Garbage): B, E
```

3. Sweeping Phase

After marking, the GC removes unmarked objects.

Before Removal

```
[A][B][C][D][E]
```

After Removal

```
[A][][C][D][ ]
```

4. Compact/Fragmentation Phase

```
[A][][C][D][ ]
[A][C][D][][ ]
```

Generational Garbage Collection Model(GGC):

- Generation 0: Short-lived objects (e.g., temporary variables).
- Generation 1: Objects promoted from Generation 0 that survived a GC cycle.
- Generation 2: Long-lived objects (e.g., static data or application-wide resources).

Runtime Exception Handling

provides a structured and unified way to handle errors through exceptions, which are objects derived from the **System.Exception** base class.

- System.NullReferenceException
- System.IndexOutOfRangeException
- System.OutOfMemoryException

Evolution of .NET

1. .NET Framework(2003-Now)
2. Mono (2004 - Now)
3. .NET Core (2016-2020)
4. .NET (2020- Now)

.NET Standard

.NET Framework(2003 - Now):

Windows-only: Built to develop Windows desktop and web applications

Close Source:- Develop and Managed by Microsoft

Key Technologies: ASP.NET, Windows Forms, ADO.NET

CLR (Common Language Runtime): Core runtime for memory management, type safety, etc.

Limitations:

- Tightly coupled to Windows

Mono

"Mono" refers to an open-source, cross-platform implementation of the .NET framework.

allows developers to write .NET applications that can run on platforms other than Windows, such as Linux, macOS, and even mobile platforms like iOS and Android.

- not an official version of microsoft.
- supports a significant portion of the .NET Framework, but it doesn’t fully support the latest .NET Core and .NET 5+ APIs.
- is geared more towards legacy compatibility, mobile (Xamarin), and game development (Unity).

Mono Runtime

core component of the **Mono** framework,

allows .NET applications to run on platforms that do not natively support the .NET runtime, such as Linux, macOS, and mobile devices.

provides the core services for executing .NET code, including garbage collection, JIT compilation, and interop with native libraries.

Mono runtime (Mono):

Mono Compiler (mcs):

Mono Class Libraries:

.NET CORE

Published in 2016

.NET Core is an open-source, cross-platform, cloud-friendly, and modular framework introduced by Microsoft as a successor to the .NET Framework.

Improvement in .NET CORE over .NET Framework

Cross-Platform Support: Runs on Windows, macOS, and Linux, enabling broader deployment and development flexibility compared to the Windows-only .NET Framework.

Open Source and Active Community: Fully open-source, with continuous contributions from Microsoft and the global developer community for rapid innovation and feature improvements.

Cloud-Native and Microservices-Friendly: Designed for containerization and cloud platforms, making it ideal for modern microservices architectures.

Modular Framework: allows developers to include only the specific libraries and packages they need through tools like NuGet, enabling lightweight, flexible, and efficient application development.

Links:

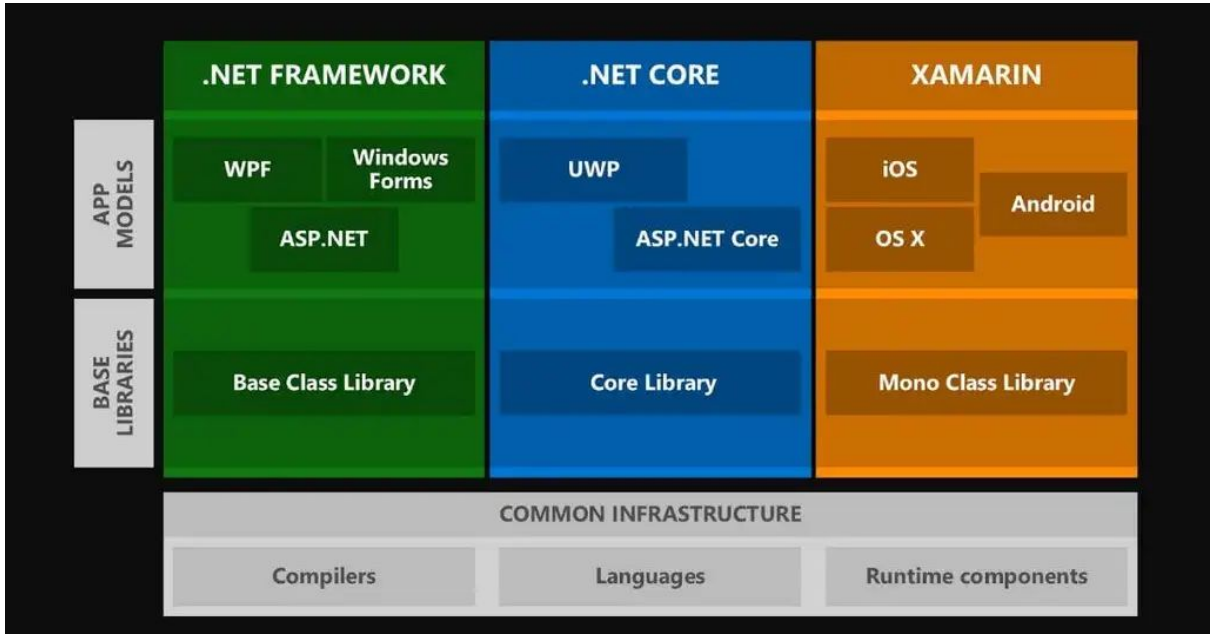
https://learn.microsoft.com/en-us/lifecycle/products/microsoft-net-and-net-core

https://versionsof.net/framework/

.NET or .NET 5

Microsoft unified the .NET ecosystem into a single platform, combining the best of .NET Framework and .NET Core and Xamarin.

Published in 2020



.NET – A unified platform

.NET or .NET 6


.NET – A unified development platform
ECOSYSTEM
TOOLS
CLOUD
Azure
WEB
ASP.NET
Blazor
DESKTOP
.NET MAUI
WPF
WinForms
MOBILE
.NET MAUI
Xamarin
GAMING
Unity
IoT
ARM32
ARM64
AI
ML.NET
.NET for
Apache Spark
NuGet
Visual Studio
GitHub
Visual Studio for Mac
Components,
tools, library
vendors
Visual Studio Code
CLI
.NET 6
COMMON BASE LIBARIES/APIs
INFRASTRUCTURE
RUNTIME COMPONENTS
COMPILERS
LANGUAGES

.NET CLI

.NET CLI is a command-line tool for building, running, and managing .NET applications.

It works across platforms: Windows, Linux, and macOS

Useful for developers who prefer terminal-based workflows, automation, or CI/CD scripting.

Common Commands:

- **dotnet new**
  - Creates a new project or file from a template (e.g., console, webapi, mvc).
- **dotnet build**
  - Compiles the app and its dependencies into a binary.
- **dotnet run**
  - Builds and runs the application (for development/testing).
- **dotnet publish**
  - Prepares the app for deployment by compiling it and placing the files into a publish folder.

New

- **Console App:** *dotnet new console -n MyConsoleApp*
- **Web API:** *dotnet new webapi -n MyWebApi*
- **MVC Web App:** *dotnet new mvc -n MyMvcApp*
- **Class Library:** *dotnet new classlib -n MyLibrary*

Useful for developers who prefer terminal-based workflows, automation, or CI/CD scripting.

Publish

- **Console App:** *dotnet publish -c Release -r win-x64 --self-contained true*

.NET Core Application

- **Console Application**: Command-line utilities.
- **Web API Application**: Backend RESTful services.
- **ASP.NET Core MVC Application:** Full-stack web apps.
- **Worker Services**: Background tasks or microservices.
- **Blazor Apps**: Interactive web UIs with C#.
- **MAUI/Xamarin**: Mobile apps using .NET

.Net Standard

.Net Standard is a specification that can be used across all .NET implementations.

It was introduced to provide a common set of APIs that would allow developers to write libraries that work across different .NET platforms, such as .NET Framework, .NET Core, and Xamarin.

Backward compatibility in .NET Standard means that libraries built for an earlier version of .NET Standard should work on platforms that support a later version of .NET Standard, but not necessarily the reverse

.NET Standard	1.0	1.1	1.2	1.3	1.4	1.5	1.6	2.0	2.1
.NET Core	1.0	1.0	1.0	1.0	1.0	1.0	1.0	2.0	3.0
.NET Framework [1]	4.5	4.5	4.5.1	4.6	4.6.1	4.6.1 [2]	4.6.1 [2]	4.6.1 [2]	N/A[3]
Mono	4.6	4.6	4.6	4.6	4.6	4.6	4.6	5.4	6.4
Xamarin.iOS	10.0	10.0	10.0	10.0	10.0	10.0	10.0	10.14	12.16
Xamarin.Mac	3.0	3.0	3.0	3.0	3.0	3.0	3.0	3.8	5.16
Xamarin.Android	7.0	7.0	7.0	7.0	7.0	7.0	7.0	8.0	10.0
Universal Windows Platform	10.0	10.0	10.0	10.0	10.0	10.0.16299	10.0.16299	10.0.16299	TBD
Unity	2018.1	2018.1	2018.1	2018.1	2018.1	2018.1	2018.1	2018.1	TBD

Feature of .NET Standard

- Cross-Platform Compatibility
- API Consistency
- Simplifies Code Sharing

.NET Standard 2.0

includes a much larger set of APIs compared to earlier versions (such as .NET Standard 1.x). It includes over 32,000 APIs, providing richer functionality and enabling more code to be shared across platforms.

NET Standard 2.0 is supported by multiple .NET platforms, including: .

- NET Core 2.0 and later
- .NET Framework 4.6.1 and later
- Xamarin (iOS, Android, and Mac)
- Mono

Basic Program Language(C#)

General-purposed, object-oriented, static typed based programming language developed by Microsoft.

It was created by Anders Hejlsberg and released in 2000.

Key Features of C#

- Object-Oriented Programming (OOP)
- Type-Safe
- Cross-Platform Development
- Versatile Application Domains

Object Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm in which real-world entities and system components in a program are represented as objects.

These objects are created from a blueprint called a class, which defines the data (properties) and actions (methods) the objects can have.

Object-Oriented Programming (OOP) in C#

OOP is a programming paradigm that **organizes** code using objects and **classes**.
It helps make programs **modular, reusable**, and **easier to maintain**.

Classes

Student Class
- Name
- ID
- Enroll()

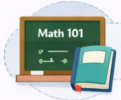
Professor Class
- Name
- Department
- Teach()

Course Class
- CourseName
- Credits
- Schedule()

Objects


Alice (Student)


Dr. Smith (Professor)



Math 101 (Course)

Objects are **Instances** of **Classes**

Modular, Reusable & Easy to Maintain!

Procedural Programming in C#

Procedural programming organizes code by dividing tasks into **separate functions**. It lacks the object-oriented features of classes and objects, leading to **more rigid, less reusable**, and **harder to maintain** code.

Procedural Code

- `void EnrollStudent()`
- `void DisplayStudent()`
- `void AssignProfessor()`
- `void DisplayProfessor()`
- `void AddCourse()`
- `void ShowCourseSchedule()`

- Separate Functions for Each Task
- Code is Tightly Coupled
- More Repetition
- Difficult to Modify

Procedural Code


Alice (Student)


Assign Professor)

Math 101 (Course)


Dr. Smith (Professor)

`void ShowCourseshSchedule()`

Rigid, Less Reusable & Harder to Maintain!

Object-Based vs Object-Oriented Programming

Object-Based Programming

Object-Oriented Programming (OOP)

	Object-Based Programming	Object-Oriented Programming (OOP)
Definition	Uses objects, but does not fully support all OOP features.	Based on **objects** and **classes**, supporting full OOP principles.
Classes	✗ Usually does not use classes.	✓ **Uses classes** to create objects.
Inheritance	✗ Not supported	✓ Supported
Polymorphism	✗ Not supported	✓ Supported
Encapsulation	⚠ Limited support	✓ **Full** support
Abstraction	⚠ Limited or none	✓ **Fully** supported
Code Reusability	→ Low	✓ **High**
Program Structure	Objects exist, but without relationships like inheritance and polymorphism and proprhism.	

Object-Based Programming

```
let student = {
  name: "Alice",
  id: 101,
  enroll: function () {
    console.log("Student enrolled");
  }
};
```

Object-Oriented Programming (OOP)

```
class Student
   public string Name,
   public int Id;
   public void Enroll() {
     Console.WriteLine("Student enrolled");
}
Student s1 = new Student();
  S1.Name = "Alice";
```

All object-oriented languages are object-based, but not all **object-based languages are object-oriented.**

Type Safety

feature of some programming languages where the type of a variable is known/differentiate at compile time.

refers to the concept of ensuring that variables, objects, and data are used in ways that are consistent with their declared types.

```
int number = 5;
string text = "Hello";
// The following will cause a compile-time error:
number = text; // Error: cannot implicitly convert type 'string' to 'int'
```

Advantage

- **Type Safety Operation & Early error Detection**

  Errors related to incompatible types are detected during compile time, before the program runs.

```
int num = 10;
string str = "123";
// This will throw an error at compile-time because you can't add a string to an int:
int result = num + str; // Compile-time error!
```

- **Improve Code Reliability & Maintainability**

  Type-safe code improves maintainability by making data types explicit and easy to understand. It optimizes performance by enforcing type checks at compile time, avoiding runtime overhead. Additionally, it supports safer refactoring, as the compiler catches type mismatches during code changes.
- **Memory Management**

  Prevents illegal memory access and reduces risks like data corruption or crashes.

Platform Support

"Platform Support" typically refers to the compatibility of C# applications with different operating systems, devices, and environments.

- Windows: Full support through .NET Framework, .NET Core, and UWP.
- macOS: Supported via .NET Core and Xamarin.
- Linux: Supported via .NET Core.
- Mobile: Supported via Xamarin (iOS/Android).
- Web: Supported via Blazor (WebAssembly) and ASP.NET Core.
- Game Development: Supported via Unity (with Mono).

Application of C#

- **Desktop Applications:**
  - build desktop applications for Windows using technologies like Windows Forms and Windows Presentation Foundation (WPF)
- **Web Applications**:
  - ASP.NET and ASP.NET Core allow developers to create dynamic web applications and services using C#
- **Mobile Applications**:
  - Xamarin, a part of the .NET ecosystem, enables C# developers to build cross-platform mobile applications for iOS and Android

- **Game Development**:
  - C# is the primary language for Unity, one of the most popular game development engines.
- **Cloud Services**:
  - C# can be used to create cloud-based applications and services using Microsoft Azure.
- **IoT (Internet of Things)**:
  - C# is used in IoT development, allowing for the creation of applications that interact with various devices and sensors.

Console Application

The Console class belongs to the System namespace.

It is used for input and output operations in console applications.

It allows programs to display messages and read user input.

Common Console Methods

- Console.Write()
- Console.WriteLine()
- Console.ReadLine()
- Console.Read()
- Console.ReadKey()
- Console.Clear()

```
using System;
using System.Security.Principal;
using System.Security.Permissions;

class RBSExample
{
   static void Main()
   {
      // Set the current user's role (simulating a user in an enterprise application)
      WindowsPrincipal currentUser = new WindowsPrincipal(WindowsIdentity.GetCurrent());

      // Check if the user belongs to the "Administrator" role
      if (currentUser.IsInRole("Administrators"))
      {
         Console.WriteLine("User is an Administrator.");
         // Allow access to an admin function
         AccessAdminFunction();
      }
```

Applied Application:

1. Xamarin [Sabin,Binayak, Ashan, Siraj] -> 25 Falgun
2. Web API [Nishit, Aayush, Aaryan, Manish] -> 3 Chaitra
3. ASP .NET Core [Sandhya,Preeti, Nabin,Biplov] ->26 Falgun
4. Windows Presentation Foundation(WPF) [Sudan,Basanta,Prabesh,Nishant] -> 25 Falgun
5. Windows Form [Roshni, Aakriti, Rakshya, Deepa] ->27 Falgun
6. Windows Communication Foundation(WCF) [Anish,Niruta,Sujan,Sandip] -> 27 Falgun

```
using System;
using System.IO;
using System.Security.Permissions;

class CASExample
{
    static void Main()
    {
        try
        {
            // Apply FileIOPermission to limit file system access
            FileIOPermission filePermission = new FileIOPermission(FileIOPermissionAccess.Read, @"C:\example.txt");
            filePermission.Demand(); // This will throw an exception if permission is not granted

            // If the permission check passes, we can proceed with the operation
            Console.WriteLine("Permission granted. Now reading the file.");
            string content = File.ReadAllText(@"C:\example.txt");
            Console.WriteLine(content);
```